# REALSENSE VISION

How Noah Sees the Sidewalk

Technical Deep Dive: Depth Camera, Edge Detection & Steering

*How the Intel RealSense depth camera, the Python vision subprocess, and the two independent edge detectors (Hough line-fit and line-scan) turn a raw depth+color frame into the single steering angle that keeps Noah on the sidewalk.*

*Generated July 01, 2026*

*lib/realsense/realsense_vision.py • Raspberry Pi 5 (16 GB)*

## Contents

# 1 Hardware & Why a Separate Process

Noah uses an **Intel RealSense depth camera** (D-series) running aligned depth + color streams at 640×480, targeting ~15 fps, mounted `camera_height_m` (0.6731 m) above the ground and pitched `camera_mount_pitch_deg` (33°) forward/down. The camera is deliberately treated as two synchronized frames — a depth image (millimeters per pixel) and a color image (BGR) — aligned to the same pixel grid so every color pixel has a matching real-world distance.

All of the camera processing — color classification, depth math, edge fitting — happens in a **separate Python subprocess** (`lib/realsense/realsense_vision.py`), not in the main Node.js process. Three reasons:

- **Library boundary:** pyrealsense2, OpenCV, and NumPy are Python-native; there is no equivalent first-class Node binding for this camera's SDK.
- **Fault isolation:** if the camera hangs, the driver crashes, or OpenCV throws, the vision subprocess dies and restarts on its own — it cannot take down mission control, steering, or the LCDs with it.
- **CPU budgeting:** vision runs at its own adaptive frame rate (see below) independent of the 250 ms mission tick, so a slow vision frame never stalls navigation.

The two processes talk over stdio: Node spawns Python with `nice -n 10` (lower CPU priority so it never starves the mission loop or motor control) and the subprocess writes one JSON object per line to stdout for every processed frame.

## Adaptive frame rate

Before each frame the subprocess samples CPU load (`psutil.cpu_percent`) and throttles its own target fps: 15 fps normally, dropping to 10 fps above `cpu_high_threshold` (70%) and 7 fps above `cpu_critical_threshold` (85%). This keeps the Pi's CPU headroom for the mission loop, MAVLink I/O, and motor control even under heavy vision load.

> **Camera reset on startup**
>
> Every time the subprocess starts it performs a hardware reset of the RealSense device (`_reset_camera`) and waits for it to re-enumerate before opening the pipeline. This clears a wedged firmware state left behind by an unclean shutdown or a previous crash — a common cause of “No device connected” errors on embedded Linux.

# 2 Data Flow: setup.json to Steering

Every tunable in this document lives in `setup.json` under the `realsense_vision` key. The path that value takes to actually reach the detector is a single, deliberate line — there is no hand-maintained subset in the middle anymore:

```
setup.json (realsense_vision: {...})
-> notip.js white_rabbit.realsense.vision_full = the RAW section, unfiltered
white_rabbit.realsense.vision = a small curated Node-side subset
-> connect_to_realsense.js start_realsense_vision()
vision_config = Object.assign({}, vision_full, { ...path fixups... })
spawn('nice', ['-n','10', python_path, script_path, JSON.stringify(vision_config)])
-> realsense_vision.py config = json.loads(sys.argv[1])
-> self.config.get("any_key", default) <- every setup.json key is live here
```

**This wasn't always true**

Two earlier hand-maintained JS objects each forwarded only a subset of `realsense_vision` to the subprocess. Keys like `edge_hough_detector`, `edge_roi_*`, `edge_line_*`, `edge_mask_threshold`, and `camera_mount_pitch_deg` were silently dropped — tuning them in setup.json did nothing because Python never received them. `connect_to_realsense.js` now forwards `vision_full` wholesale, so any key added to setup.json's `realsense_vision` section takes effect the next time the vision process restarts. Do not reintroduce a subset.

## The return path

Results flow back the other way: Python emits one `path_detection` JSON message per processed frame → Node's stdout handler splits it on newlines → `realsense_message_handler.js` parses it and writes the fields onto `white_rabbit.realsense.path_detection` → `follow_the_yellow_brick_road.js` reads that struct every 250 ms mission tick to compute a steering angle. Rover pitch and roll are streamed the opposite direction at 10 Hz (`start_pitch_stream`) so the depth math in Python can correct for the rover rocking over bumps and off-camber pavement.

# 3 Per-Frame Pipeline Overview

`process_frames()` runs once per accepted frame and does two independent jobs: **detect_path** (the sidewalk edges — this document's focus) and **detect_objects** (obstacles, §9). Both work from the same aligned depth + color pair and the same intrinsics.

- **Align:** `rs.align(rs.stream.color)` maps every depth pixel onto the color pixel grid so a single (x, y) index reads both a color and a real-world distance.
- **Crop the ROI:** only the lower part of the frame is used for path detection — rows from `edge_roi_top_frac` (0.4) to `edge_roi_bottom_frac` (1) of the image height. This excludes sky, distant buildings, and anything above the ground plane before any classification runs.
- **Classify walkable ground** from color + depth (§4).
- **Filter to real ground** using 3-D geometry, not just appearance (§5).
- **Fit independent left/right edges** using one of two detectors, selected by `edge_hough_detector` (§6, §7).
- **Assemble the result:** smooth, cache, choose a side, compute the steering angle (§8).

Everything downstream of the ROI crop operates in **meters**, not pixels, using the color stream's intrinsics (`fx, fy, ppx, ppy`) to deproject any pixel + depth reading into a camera-frame 3-D point: `cam_X = (x_px - ppx) * depth_m / fx`, `cam_Y = (y_px - ppy) * depth_m / fy`. That 3-D point is then rotated by the rover's current roll and pitch (streamed live from Node) to recover a true horizontal forward distance and lateral offset — the same un-roll/un-pitch transform is reused in the ground mask filter, the centerline, the edge-clearance check, and object detection, so a pothole or a banked driveway can't silently corrupt one calculation while leaving another correct.

# 4 Building the Walkable Ground Mask

`_build_simple_ground_mask()` turns the cropped color ROI into a binary “walkable / not-walkable” mask using color alone — depth-based ground validation happens afterward, in §5.

### Step by step

- **CLAHE illumination normalization** (`simple_edge_clahe_enabled`, default on): the ROI is converted to LAB color space and contrast-limited adaptive histogram equalization is applied to the lightness channel. This flattens dappled tree-shadow patterns on concrete so a shadow edge doesn't get misread as a sidewalk boundary.
- **Adaptive concrete threshold:** the value floor for “concrete-bright” pixels is not a fixed number — it's `max(light_min, val_min, mean_brightness × val_mean_frac)`, so the same detector works in both bright sun and overcast light.
- **Exclusion masks** in HSV subtract anything that looks like grass (green hue band), mulch/bark/soil (brown-orange hue band), or near-black shadow/asphalt-crack pixels from the concrete mask.
- **Cleanup:** median blur removes salt-and-pepper noise; morphological open then close fills small holes and removes small false-positive specks; a connected-components pass drops any surviving blob smaller than `simple_edge_component_min_area` pixels.
- **Keep the centered blob:** `_select_center_component()` keeps only the walkable blob whose centroid is horizontally closest to the image center — the sidewalk Noah is standing on, not a driveway or patch of concrete off to one side.

The output is a single 0/255 mask, plus the raw green mask (reused later to bias edge classification away from grass boundaries).

# 5 The Ground-Plane Filter & Perspective Check

Color alone is fooled easily: a gray wall, a parked car's shadow, or a raised concrete planter can read as “walkable” by hue and brightness. Two independent geometric checks guard against this before any edge is trusted.

## TRON-grid ground-plane filter

`_apply_ground_grid_filter()` deprojects every masked pixel into a rover-horizontal world frame (`world_X` = lateral, `world_Z` = forward, `world_Y` = height above the expected ground plane, using `camera_height_m` as the zero reference). It then lays a grid of `ground_grid_cell_m` (0.25 m) cells over the X/Z ground plane. Any cell with enough samples (`ground_grid_min_samples`) where most points sit off the ground plane by more than `ground_height_tol_m` (0.1 m) is flagged “not ground” and every mask pixel in it is removed — walls, raised beds, bushes, fences, parked cars, and grass berms all fail this test even if they were misclassified as walkable by color. Cells with too few depth samples to judge are left alone (benefit of the doubt), so a real sidewalk with imperfect depth coverage still survives.

## Perspective-narrowing validation

A real sidewalk of roughly constant width projects to a pixel width that shrinks with distance (perspective: pixel width ∝ fx / depth). `_validate_perspective_narrowing()` measures the walkable band's pixel width in several horizontal bands, converts each to a real depth, and checks the near/far ratio against `perspective_min_narrowing_ratio`. Noah's setup.json currently loosens this to **0.85×** — below 1.0, so the near band only has to be at least 85% as wide as the far band; a frame can even get very slightly WIDER with distance and still pass. This is a deliberately weaker check than the 105%-stricter code default, likely tuned to tolerate this camera's 33° forward mount pitch, which flattens the apparent perspective taper at close range. A frame that fails this check is rejected as `perspective_invalid`.

> **Failing gracefully, not silently**
>
> When perspective validation fails, the detector does NOT zero out both edges. It still serves each side's last-known cached position independently (subject to the TTL in §8) so a momentary bad frame on one side doesn't blank a perfectly good detection on the other.

# 6 Edge Detector #1 — Hough Line-Fit (current default)

`_detect_edges_hough()` is the live detector (`edge_hough_detector` defaults **true**). It is lane-departure-style: instead of finding one walkable blob and reading both of its sides (which couples the two edges together — losing sight of one side used to blank the other), it fits ONE line per side, independently, from candidate edge pixels.

### 1. Candidate edge pixels

Two sources are OR'd together into a single edge image — deliberately **not** raw Canny across the whole frame, because Canny alone fires on walls, furniture, and shadows that have nothing to do with the ground:

- **Color-class boundary:** a morphological gradient of the walkable mask from §4 — the boundary between walkable and not-walkable pixels.
- **Depth drop-off:** a horizontal derivative of the depth image; any adjacent-pixel jump greater than `dropoff_min_depth_jump_m` (0.08 m) is marked as an edge pixel — this is how a curb or a grass step gets caught even when color alone doesn't clearly separate it.

### 2. Hough line segments

`cv2.HoughLinesP` extracts straight line segments from the combined edge image (`edge_line_hough_threshold`, `edge_line_min_len_px`, `edge_line_max_gap_px` control sensitivity).

### 3. One independent fit per side

- Each segment's steepness is checked against `edge_line_min_abs_slope` — near-horizontal segments (the horizon, pavement cracks) are dropped as clutter.
- Each surviving segment is assigned to **left** or **right** purely by which side of image-center its lowest (nearest) endpoint falls on.
- Per side, `np.polyfit` fits a single line `x = a·y + b`, weighted by each segment's pixel length, giving one stable line per side even when made of several short segments.

### 4. Nearest valid ground point per side

The fitted line is walked from the bottom of the ROI upward (nearest ground first); the first row where the line is in-frame AND has valid depth becomes that side's reported edge point. The line is fit over the whole ROI for stability, but only the closest point is reported — that's the point the steering formula cares about.

### Confidence

Confidence blends three independent signals: **support** (how much total segment length backed the fit), **fit quality** (how tightly the segments agree — low residual standard deviation), and **distance weighting** (a close edge is geometrically more reliable than a far one, fading from full weight at `edge_distance_full_conf_m` to zero weight at `edge_distance_zero_conf_m`). When both edges are seen at a plausible sidewalk width (0.4–2.5 m apart), both confidences get a corroboration boost (`edge_both_seen_conf_boost`) — driving straight down the middle of a narrow sidewalk shouldn't read as low-confidence just because each individual edge line is short.

**Losing one edge never blanks the other**

Because the left and right lines are fit from completely separate segment pools, a curb leaving the camera's field of view on one side has zero effect on the fit for the other side. This was the specific bug the Hough detector was built to fix — the older blob-based approach (§7) found ONE walkable region and read both of its edges from it, so losing sight of either physical edge could null both reported edges at once.

# 7 Edge Detector #2 — Line-Scan (independent bands)

`_detect_path_from_lines()` is the earlier detector, still selectable by setting `edge_hough_detector: false` and `edge_lines_only: true` — useful as an A/B fallback.

- The ROI is split into `edge_line_bands` (6) horizontal bands, near to far.
- In each band, `find_independent_edges()` scans the walkable-mask column scores outward from the walkable column nearest image-center, independently in each direction, until it exits the walkable region or hits the image border. A side that runs off the image border is treated as out-of-frame (returns None) rather than a false edge.
- Each band's left/right pixel hit is deprojected to a 3-D point exactly as in §6; across all bands, the **nearest** valid detection per side (smallest forward distance) is kept.
- Confidence here scales with how much of the local mask run backs the detected pixel (longer run → more confident), rather than the Hough detector's support/fit-quality/distance blend.

This detector still anchors each side's scan to the walkable column nearest the image center before scanning outward independently in each direction — the same anti-coupling technique `find_independent_edges()` uses in the Hough path's supporting code, so losing one edge does not blank the other here either. Both detectors funnel into the same `_assemble_edge_result()` (§8), so the LCD, message handler, and steering code are completely unaware of which detector is active.

# 8 Shared Edge Assembly: Smoothing, Caching, Selection

Both detectors hand their raw left/right observations to `_assemble_edge_result()`, the single place that turns "what did this frame see" into "what should the rover do."

### EMA smoothing

Each side's lateral position is smoothed with an exponential moving average (`edge_ema_alpha`, 0.45 — 45% new value / 55% history, roughly a 2.2-frame time constant at 15 fps). Only the lateral (X) component is smoothed; forward distance (Y) is left raw, because smoothing Y would introduce lag that inflates the apparent forward distance and can silently block corrections that check against a maximum forward distance.

### Per-side last-known cache (TTL)

If a side isn't detected this frame, its previous observation is reused for up to `edge_known_ttl_ms` (5000 ms), tagged as `known=true` with an age. Past the TTL it reports as not-seen. This is what lets the rover ride through a brief occlusion (a pedestrian, a shadow flicker, a momentary bad frame) without the correction snapping to zero and back.

### Validity filters

- A detected edge closer than `edge_min_lateral_m` (0.25 m) to the camera's centerline is discarded — too close to be a real sidewalk boundary, more likely noise.
- A "left" edge reported on the positive (right) side of center, or a "right" edge reported on the negative (left) side, is discarded — a basic sanity check against a detector bug swapping sides.

### Choosing what to steer on

- **Both edges known** → steer to the midpoint (`use = "center"`); target offset is the mean of both sides' signed lateral position.
- **One edge known** → hold `edge_side_offset_m` (0.5 m / ~1.6 ft) off it: to the right of a left edge, or to the left of a right edge.
- **Neither known** → `edge_guidance_valid = false`; the JS side latches the last correction briefly, then fades to GPS-only (§10).

The final steering signal is one number: `x_angle_deg = atan2(-target_offset, forward_distance)` — the angle from the camera's boresight to the point the rover should be driving toward. Positive means the target is to the right.

# 9 Object Detection & Threat Zones

`detect_objects()` runs on every frame alongside path detection, using the same pitch/roll-corrected deprojection math but over the **full** depth frame (downsampled 4× for CPU headroom, with intrinsics scaled to match).

- Every pixel is deprojected into the rover-horizontal world frame; a pixel counts as a candidate obstacle if its height above ground is between `object_min_height_m` (0.127 m / ~5 in) and 2.5 m — tall enough to matter, short enough to still be an obstacle rather than a tree canopy.
- Morphological close/open cleans the obstacle mask, then `cv2.connectedComponentsWithStats` clusters it into discrete objects, discarding clusters below `object_min_area_px`.
- Each object reports a **clock-face bearing** (12 = straight ahead) computed from `atan2(lateral, near_distance)`, a distance (20th-percentile depth, so the near edge of the object is reported rather than its center), height, width, and a **threat level**: `high` if it's within the rover's track width and closer than 1.5 rover-lengths, `medium` if either condition alone holds, otherwise `low`.

On the Node side, `realsense_message_handler.js` maps each object's clock bearing onto camera zones 10–11–12–1–2 and applies a **1-second confirmation filter** per zone before lighting it red or yellow — a single-frame flicker cannot trigger avoidance or an emergency stop; the same threat level has to persist for a full second first. A zone clears back to green immediately once the threat is gone, so the rover is never stuck waiting out a hold-time on the all-clear side.

# 10 From Camera to Wheels: the Steering Consumer

`lib/yellow_brick_road/follow_the_yellow_brick_road.js` is the only place the vision output actually turns into motor commands, once per 250 ms mission tick.

- **Confidence gate:** a raw edge reading below `confidence_threshold` (0.45) is discarded before it ever reaches the steering formula — this is what stops a far-away, low-confidence corner read from producing a large, spurious steering spike.
- **Steering angle:** with both edges accepted, the code steers toward their midpoint via `atan2(centerline_x, centerline_y)`; with only one accepted, the null side contributes zero to the average, which (since scaling both terms by the same factor doesn't change an atan2 ratio) still steers along that single edge's bearing.
- **Non-linear steering tune:** the raw angle is scaled by a tune factor that ramps from 0.50 at small angles up to 1.00 above 18° — gentle corrections are damped so the rover doesn't hunt on straight sidewalk, while sharp corrections get full authority.
- **Motor speed** is driven by the average of the two edge confidences — lower confidence means more caution, giving the vision pipeline more time (and more frames) to reacquire the edge.
- The final angle is clamped to ±20° and handed to `calc_steering_and_rpm()`, which produces the four individual servo angles and wheel RPMs sent to the servos and motor drivers.

> **Edge guidance is gated, not always-on**
>
> This whole pipeline only steers the rover when `mission.sidewalk_follow_active` is on — toggled by passing a GATE waypoint (a route turn ≥ 90°, or two waypoints placed within 1 m of each other). Outside the gated sidewalk section — driveways, roads, the area near the dock — GPS waypoint following runs alone and camera steering is suppressed entirely.

# 11 Tunable Parameters Reference

All of the following live under `realsense_vision` in `setup.json` and reach the Python subprocess wholesale (§2). Values below are read live from setup.json when this document is generated — not hand-typed — so they always match what's actually configured on Noah. Per project rule, any change here must also be mirrored into `setup_example.json`.

| Setting | As Configured | What it controls |
|---|---|---|
| **edge_hough_detector** | true | Selects the Hough line-fit detector (§6) over the line-scan detector (§7) |
| **edge_lookahead_m** | 0.45 m (~1.5 ft) | How far ahead guidance aims to read the sidewalk edge |
| **edge_side_offset_m** | 0.5 m (~1.6 ft) | Lateral gap held off a single visible edge |
| **confidence_threshold** | 0.45 | Raw edge confidence floor before a reading reaches steering (Node side) |
| **edge_ema_alpha** | 0.45 | EMA smoothing weight on edge lateral position (higher = less smoothing) |
| **edge_min_lateral_m** | 0.25 m | Detections closer than this to the centerline are discarded as noise |
| **edge_known_ttl_ms** | 5000 ms | How long a lost edge's last-known position stays valid |
| **edge_roi_top_frac / bottom_frac** | 0.4 / 1 | Vertical crop of the frame used for ground/edge detection |
| **edge_line_canny_low / high** | 45 / 130 | Canny thresholds feeding the color-boundary edge image (Hough path) |
| **edge_line_hough_threshold** | 30 | HoughLinesP vote threshold — lower finds more, noisier lines |
| **edge_line_min_len_px / max_gap_px** | 30 / 20 | Minimum segment length / max gap HoughLinesP will bridge |
| **edge_line_min_abs_slope** | 0.25 | Rejects near-horizontal Hough segments (horizon, cracks) as clutter |
| **dropoff_min_depth_jump_m** | 0.08 m | Depth discontinuity that counts as a curb/grass drop-off edge |
| **camera_height_m** | 0.6731 m | RealSense mount height above ground — zero reference for ground-plane math |
| **camera_mount_pitch_deg** | 33° | Static forward mount tilt; subtracted from live rover body pitch |
| **ground_height_tol_m** | 0.1 m | Height tolerance for the TRON-grid ground-plane filter (§5) |
| **ground_grid_cell_m** | 0.25 m | Cell size of the ground-plane classification grid |
| **perspective_min_narrowing_ratio** | 0.85 | Required near/far pixel-width ratio for perspective validation (§5) |
| **object_emergency_stop_m** | 1 m | In-path high-threat distance that triggers an immediate stop |
| **object_min_height_m** | 0.127 m (~5 in) | Minimum height above ground counted as an obstacle |

| Setting | As Configured | What it controls |
|---|---|---|
| **fps_normal / fps_high_cpu / fps_critical_cpu** | 15 / 10 / 7 | Adaptive frame-rate ladder keyed to Pi CPU load |

---

*For the operator-facing summary of edge guidance, avoidance, and LCD readouts, see* `noah_system_manual.pdf` *§10. For deeper architecture notes and the project's living memory, see CLAUDE.md and .claude/memory/ in the repository.*